

Chapter 15: Ridges and Valleys

Prof. Dr.-Ing. Thomas Schultz

URL: <http://cg.cs.uni-bonn.de/schultz/>

E-Mail: schultz@cs.uni-bonn.de

Office: Friedrich-Hirzebruch-Allee 6, Room 2.117

July 8/14, 2026

Section 15.1:

Introduction and Watershed Definition

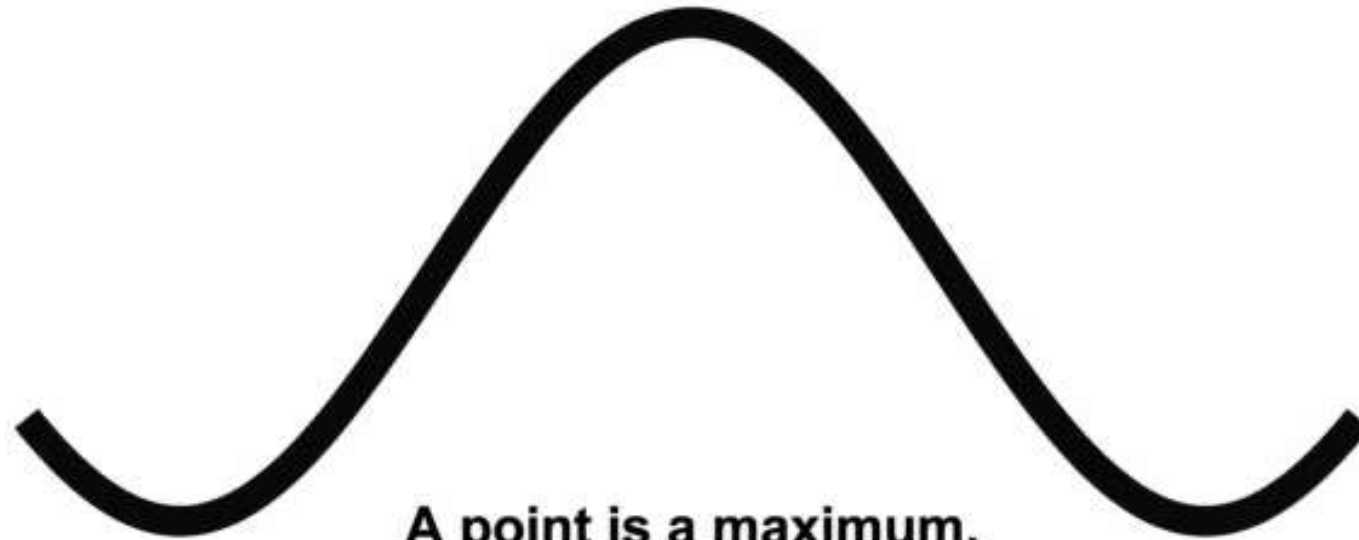
Definition of Ridges and Valleys: Intuition

Definitions by Merriam-Webster:

- ridge = “an elongate crest”
- crest = “top line of mountain or hill”

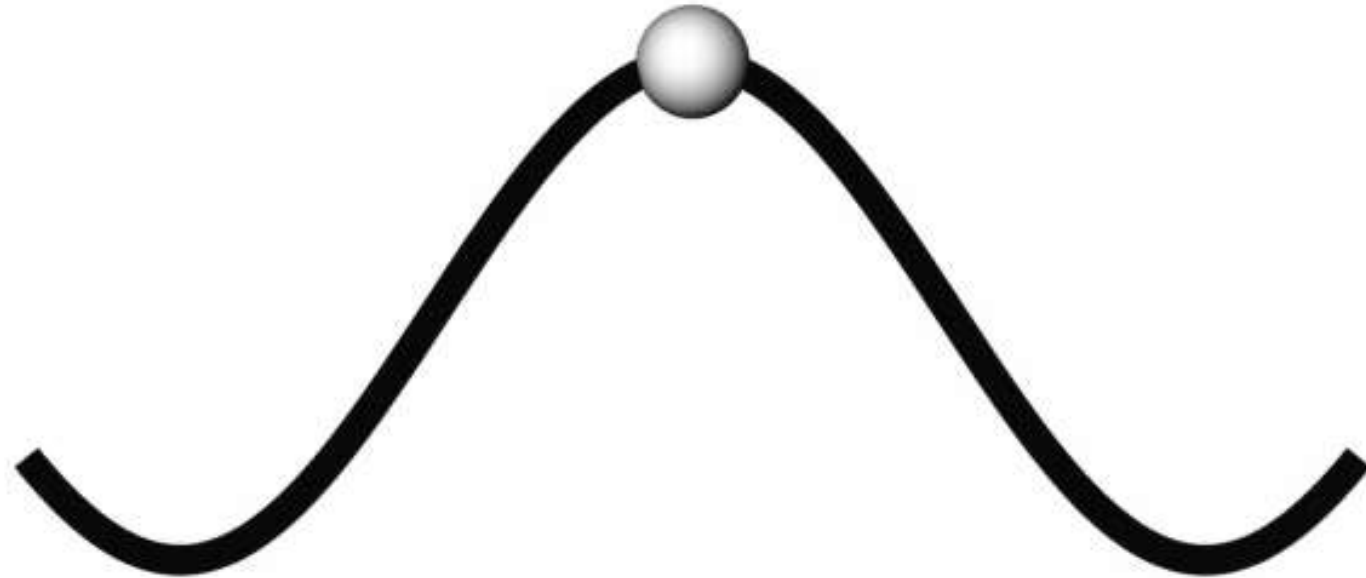


Intuition behind extremal lines



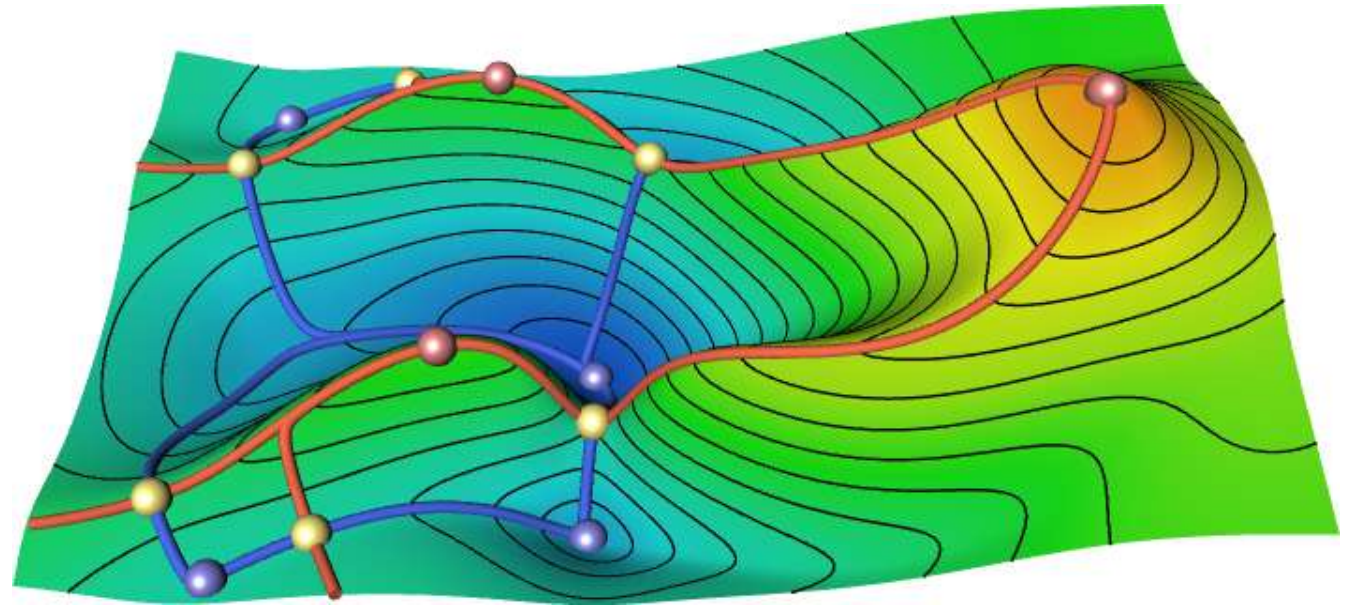
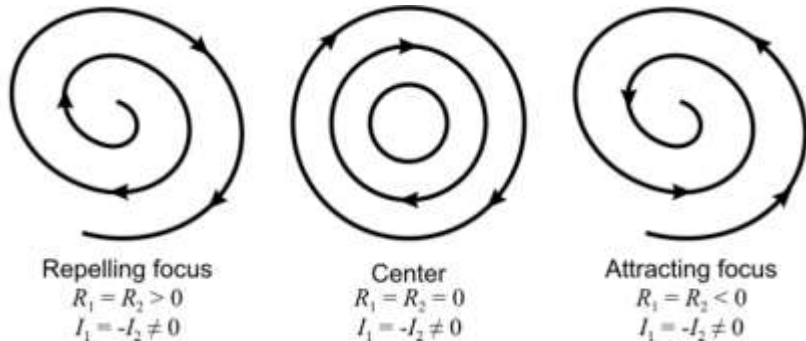
**A point is a maximum,
if its left/right neighborhood
assembles into different minima
when integrated along the gradient.**

Intuition behind extremal lines



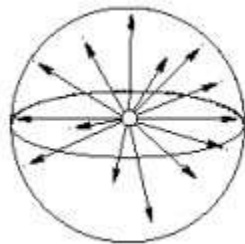
Extraction of Watersheds

- Ridges and valleys according to the watershed definition can be found by topological analysis of the gradient vector field
 - Sources / sinks are local minima / maxima
 - Saddle points are saddles
 - Separatrices are watersheds
 - *Quiz:* What about foci or center?

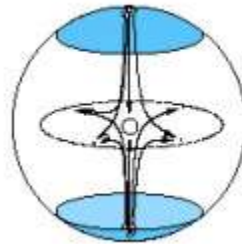


Extremal Structures of a 3D Scalar Field

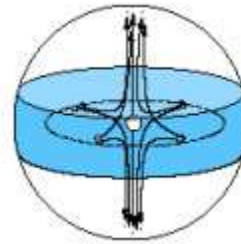
- The extremal structures of a 3D scalar field are:
 - minima and maxima points (extrema)
 - minimal and maximal lines (extremal lines)
 - extremal surfaces: surfaces of minimal/maximal scalar data value
 - Note: “Minimal surface” often denotes a surface with minimal area, not an extremal surface in our sense
- Topological analysis of (3D) gradient field



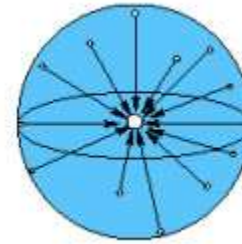
Minimum



1-Saddle



2-Saddle



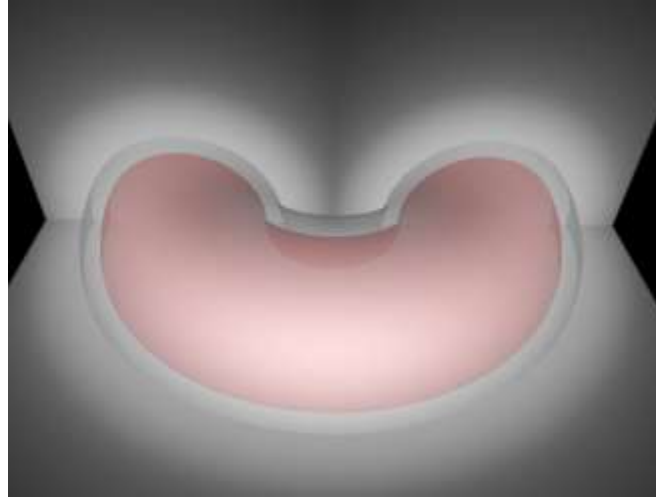
Maximum

Ridges vs. Isosurfaces

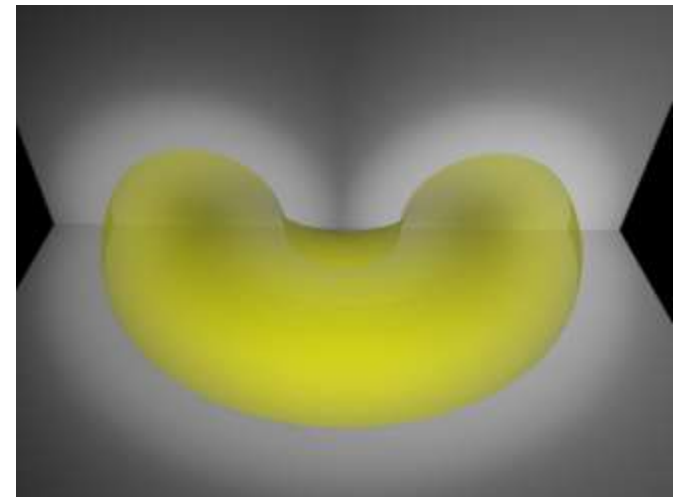
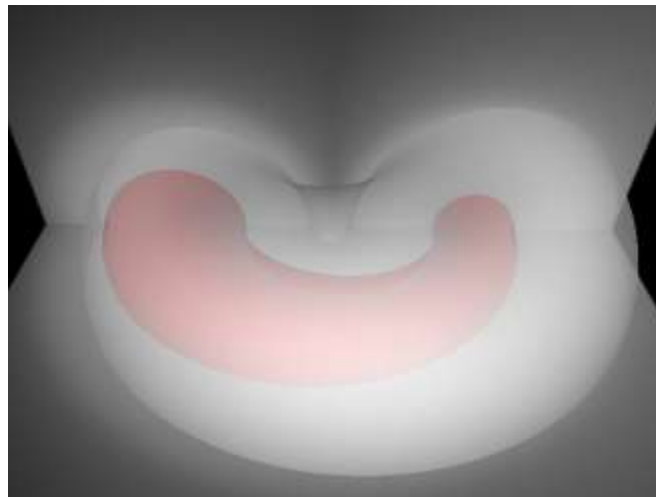
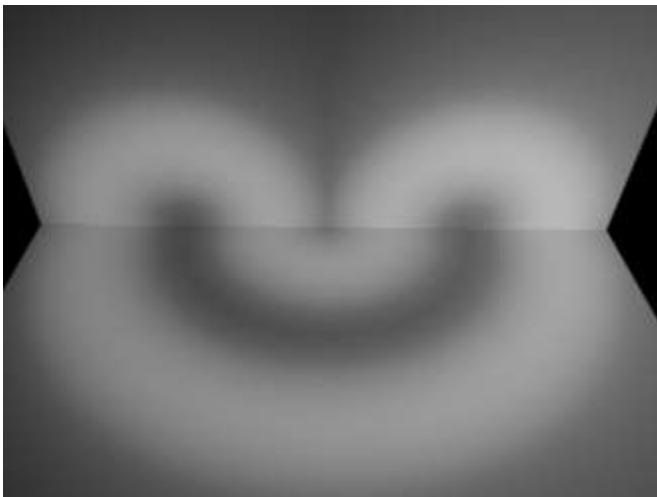
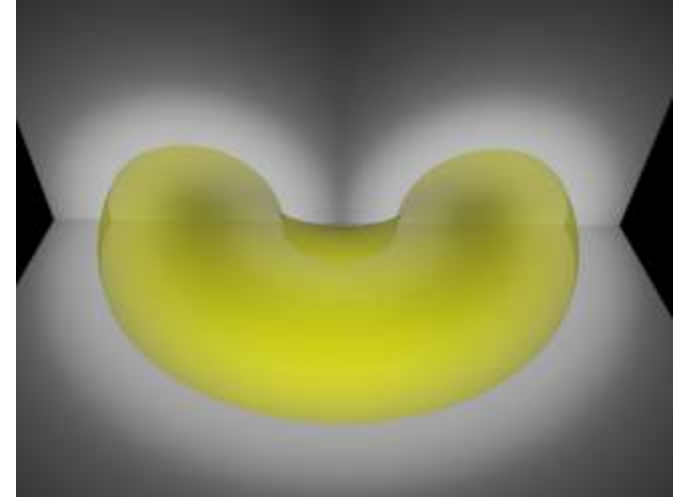
Dataset



Isosurface



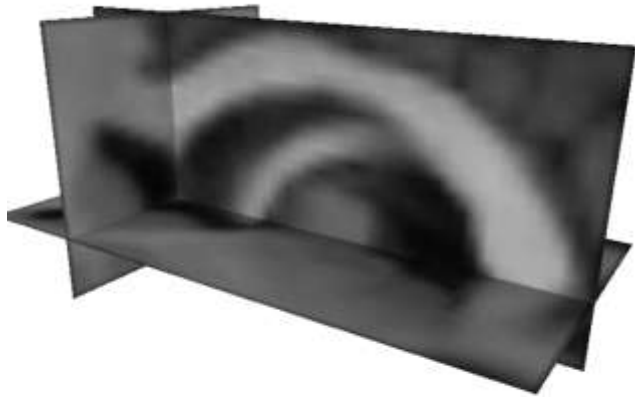
Ridge



Contours vs. Skeletons

Often:

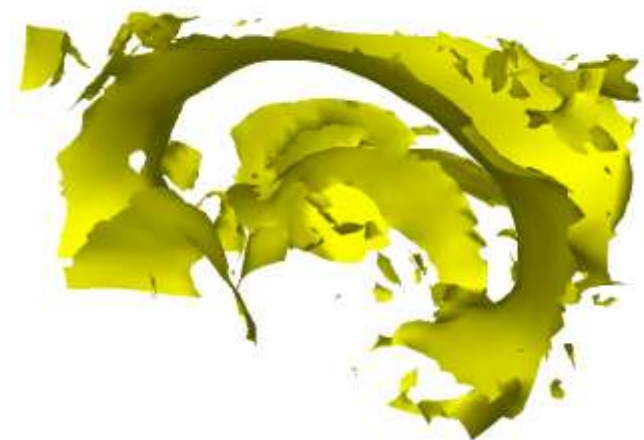
- **Isosurfaces** extract **contours**
- **Ridges and Valleys** extract **cores / skeletons**
 - Sometimes, after some transformation of the field
 - smoothing, “medialness” / “vesselness” measures



Data



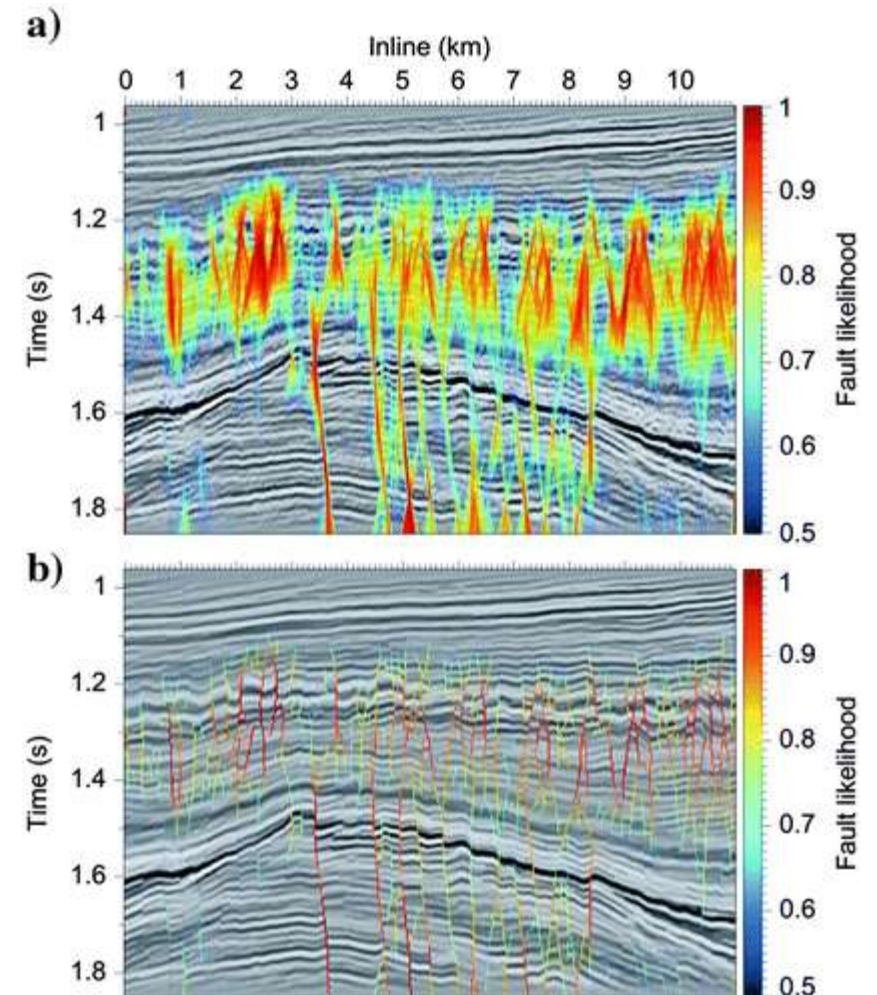
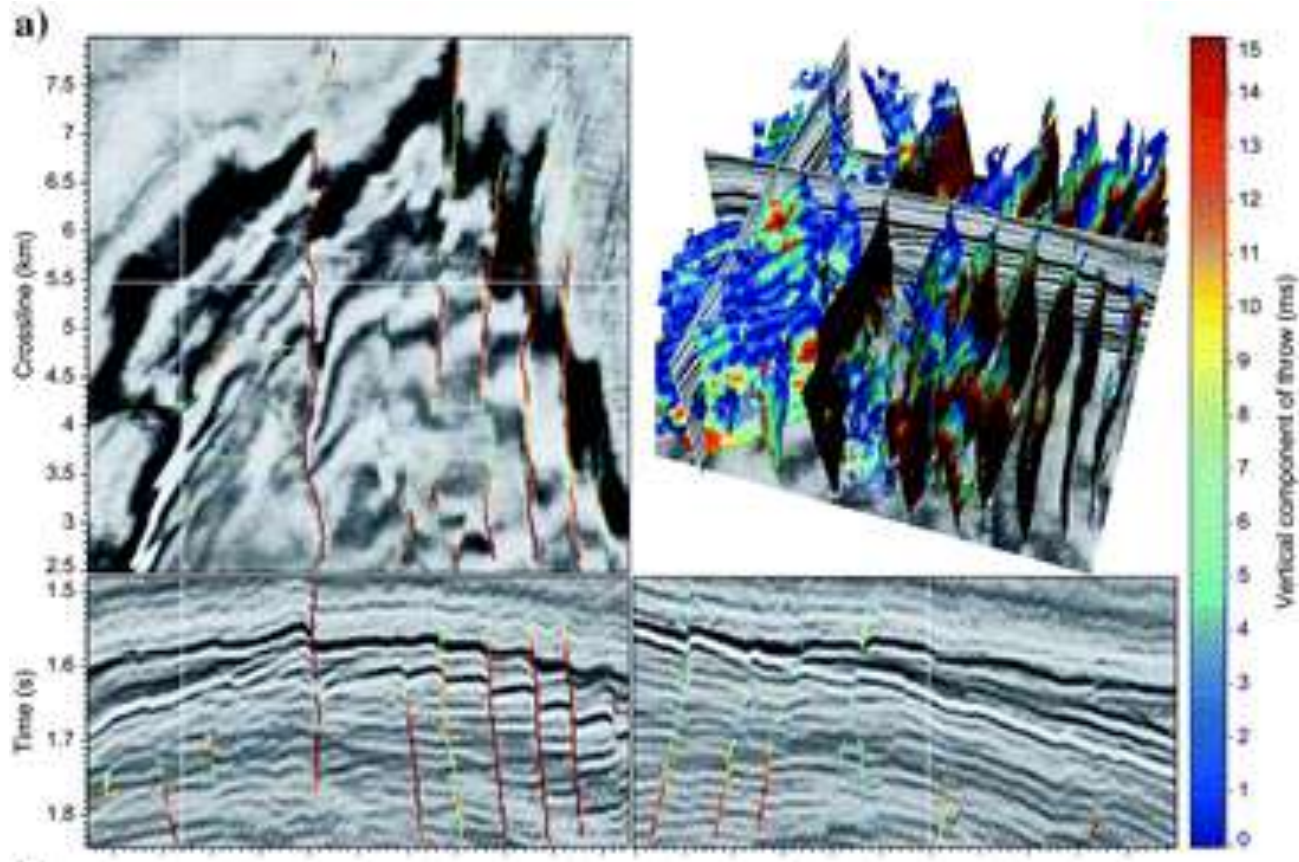
Isosurface



Ridge Surface

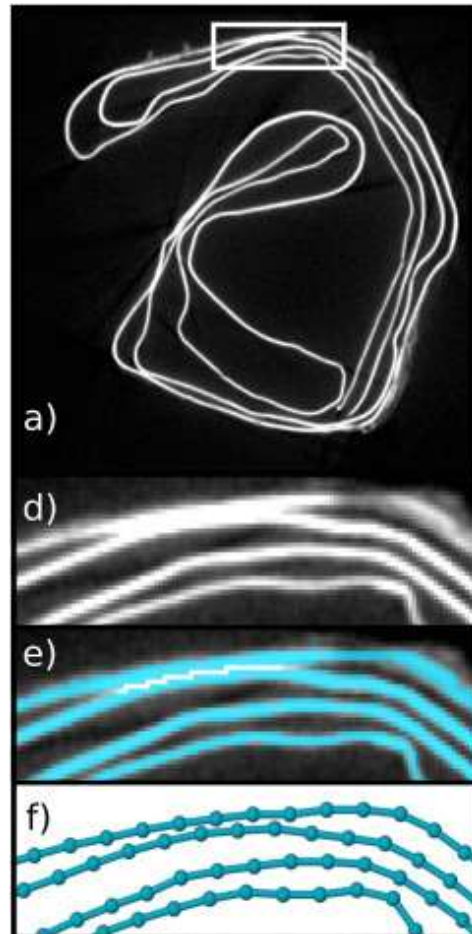
Example: Fault Surfaces in Geology

- [Hale et al. 2013]: Fault surfaces as ridges in fault likelihood computed from 3D seismic images



Example: Virtual Unfolding

- [Baum et al. 2021]: Virtual unfolding of the Jerash Silver Scroll (749 AD, discovered in 2014)

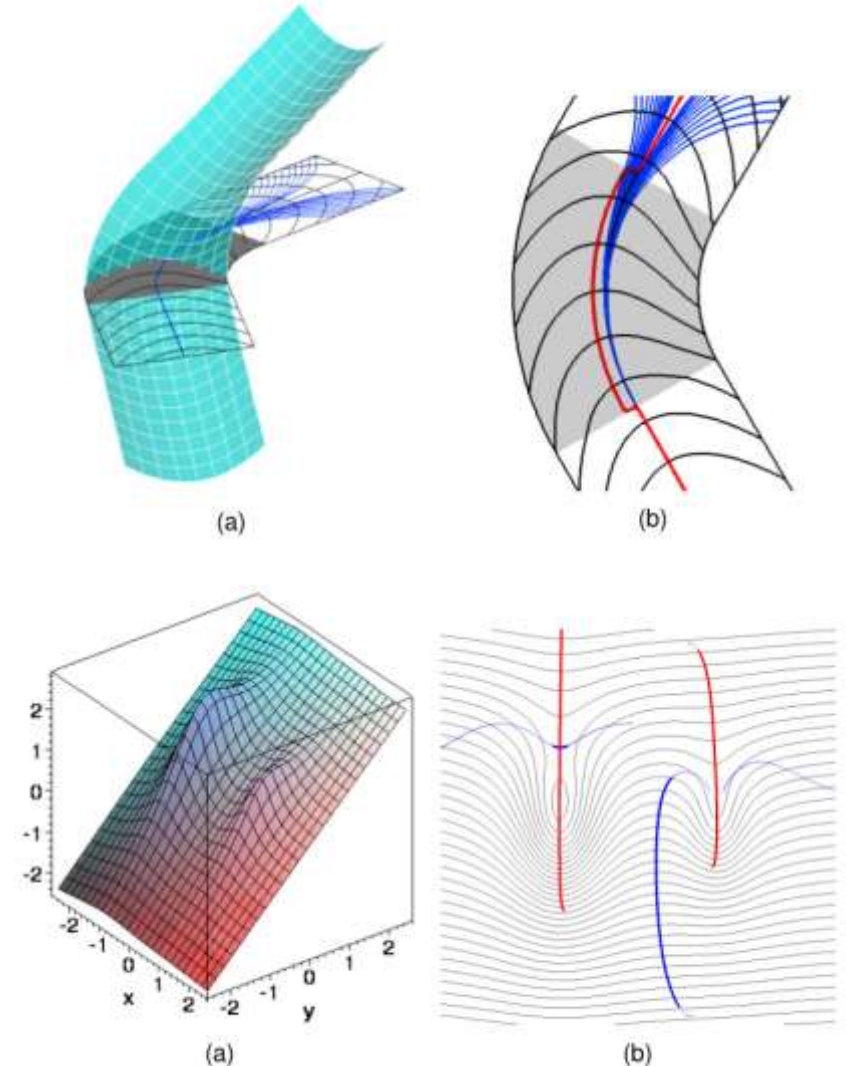


Section 15.2:

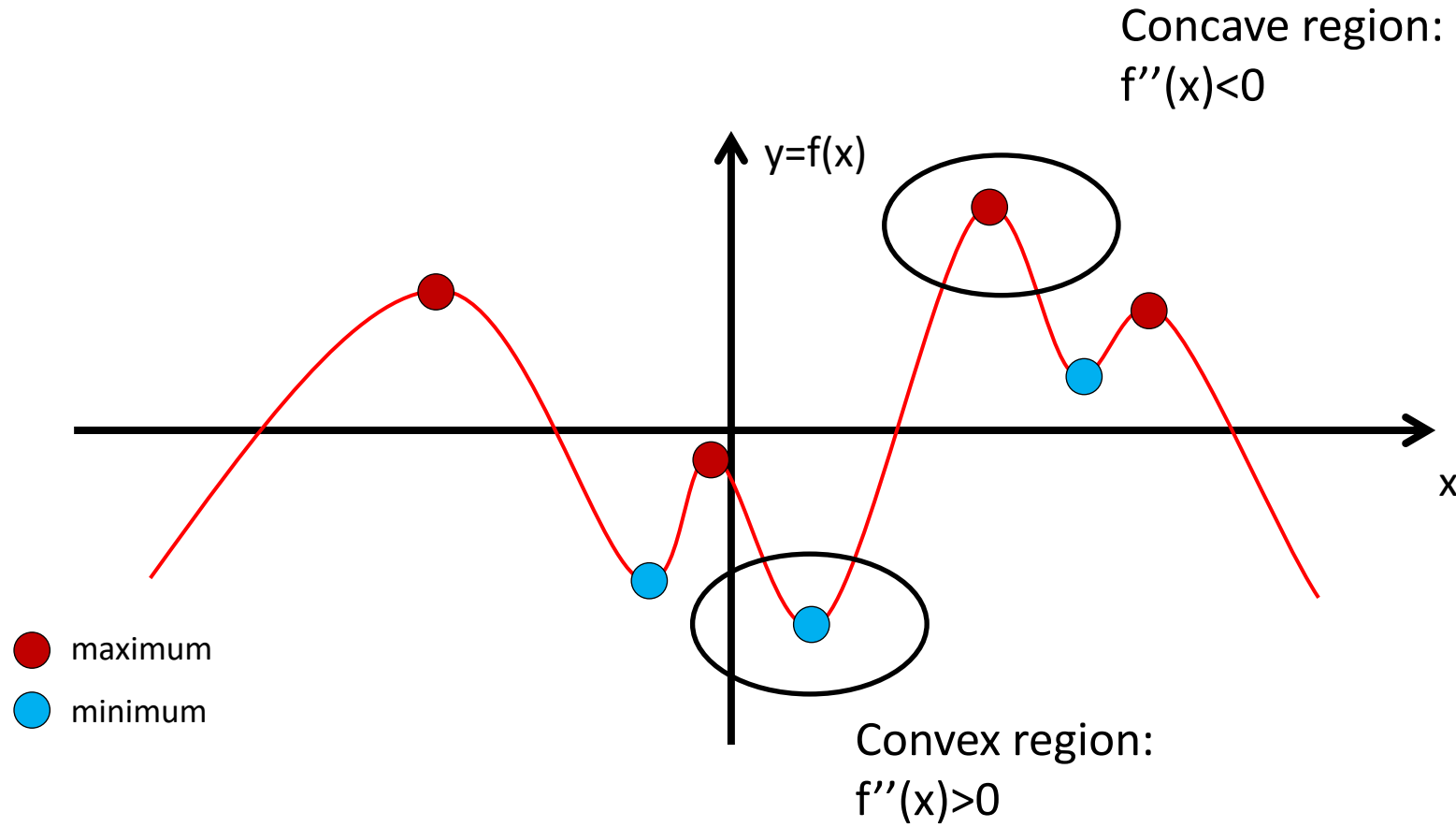
Height Ridges and Valleys

Watersheds vs. Height Ridges

- **Watersheds** are global features
 - cannot decide if a point belongs to a separatrix just by looking at its neighborhood
- Alternative: **Height ridges**
 - *benefit*: local feature definition
 - often provides similar results in practice
- *Note*: Similar to vortex cores, there is no unique “correct” formalization
 - see examples on the right



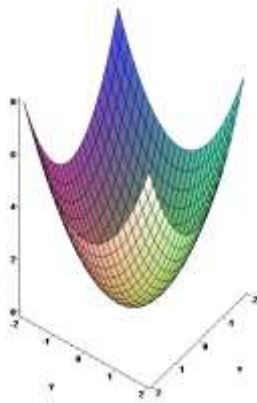
Reminder: Curvature in 1D



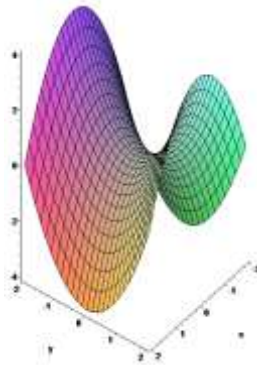
Reminder: Curvature in 2D

Surfaces have two *principal curvatures* whose signs may or may not agree.

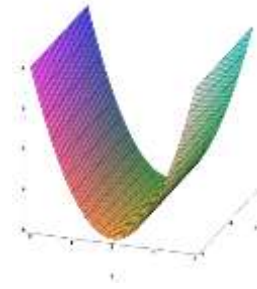
Examples:



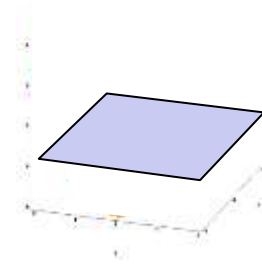
$$\kappa_i > 0$$



$$\kappa_0 > 0, \kappa_1 < 0$$



$$\kappa_0 = 0, \kappa_1 > 0$$



$$\kappa_0 = 0, \kappa_1 = 0$$

...

Eigenvectors and -values of Hessian Matrix

- **Reminder:** Since Hessian matrix is real and symmetric,
 - its eigenvectors $\mathbf{e}_i$ are orthogonal
 - can serve as a basis of the domain
 - its eigenvalues λ_i are real
 - $\lambda_i < 0$: function is concave in direction $\mathbf{e}_i$
 - $\lambda_i > 0$: function is convex in direction $\mathbf{e}_i$

Definition: Height Ridge Lines in 2D

Definition from [Eberly 1996]:

- Assume a 2D scalar field with gradient $\mathbf{g}$ and Hessian $\mathbf{H}$ ($\mathbf{e}_i / \lambda_i$ s.t. $\lambda_1 \geq \lambda_2$)
- **Local maxima** (isolated points) given by $\|\mathbf{g}\| = 0, \lambda_i < 0$
- To get **ridge lines**, only ask for vanishing $\mathbf{g}$ in direction of strongest concavity

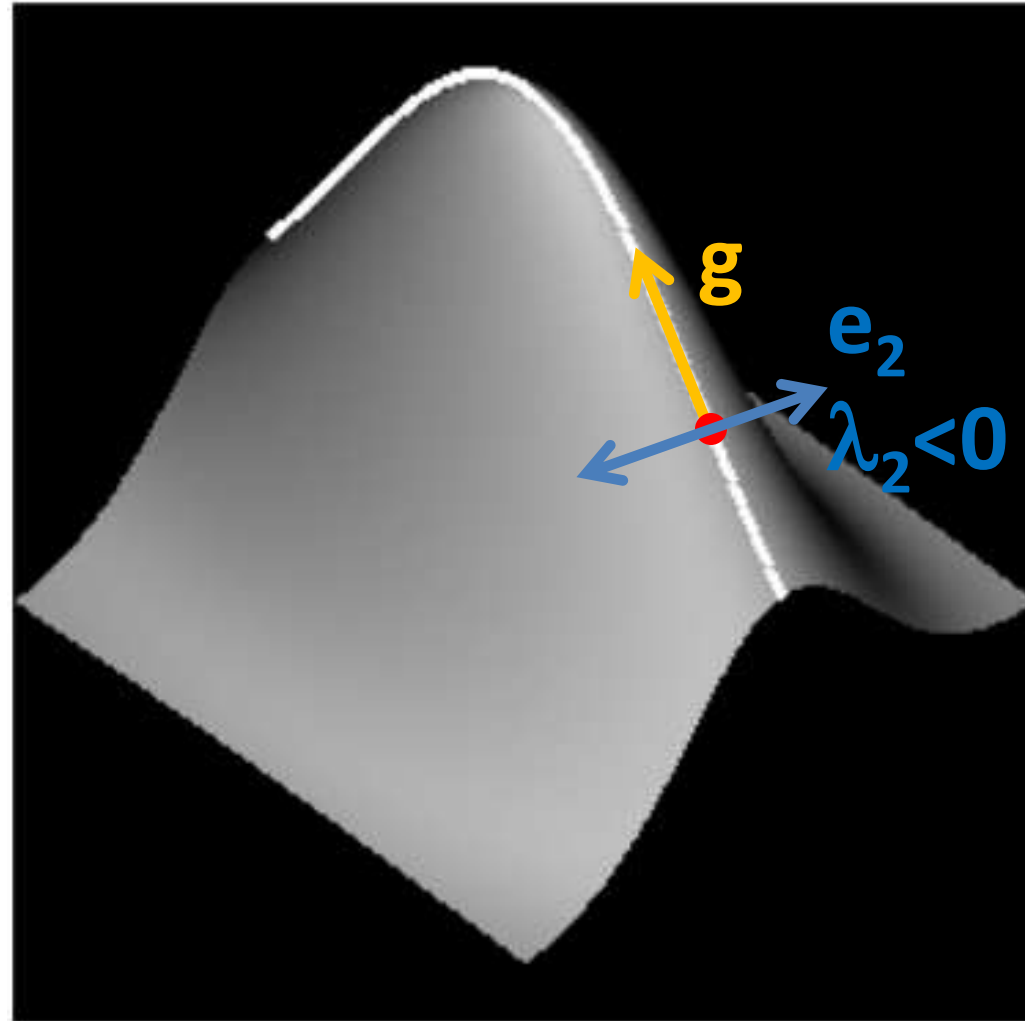
$$\mathbf{g} \cdot \mathbf{e}_2 = 0 \wedge \lambda_2 < 0$$

- **Valleys** are defined in complete analogy

$$\mathbf{g} \cdot \mathbf{e}_1 = 0 \wedge \lambda_1 > 0$$

- Ridges + Valleys are collectively called **creases**

Illustration: Height Ridge Lines in 2D



- On Ridge Line

Height Ridge Lines and Surfaces in 3D

- Assume a 3D scalar field with gradient $\mathbf{g}$ and Hessian $\mathbf{H}$ ($\mathbf{e}_i / \lambda_i$ s.t. $\lambda_1 \geq \lambda_2 \geq \lambda_3$)
- At a **local maximum** (point), $\|\mathbf{g}\| = 0$, $\lambda_i < 0$
- Asking for vanishing $\mathbf{g}$ in direction of strongest concavity leads to a **ridge surface**

$$\mathbf{g} \cdot \mathbf{e}_3 = 0 \wedge \lambda_3 < 0$$

- To get a **ridge line**, ask for vanishing $\mathbf{g}$ in *plane* of strongest concavity

$$\mathbf{g} \cdot \mathbf{e}_3 = 0 \wedge \mathbf{g} \cdot \mathbf{e}_2 = 0 \wedge \lambda_3 < 0 \wedge \lambda_2 < 0$$

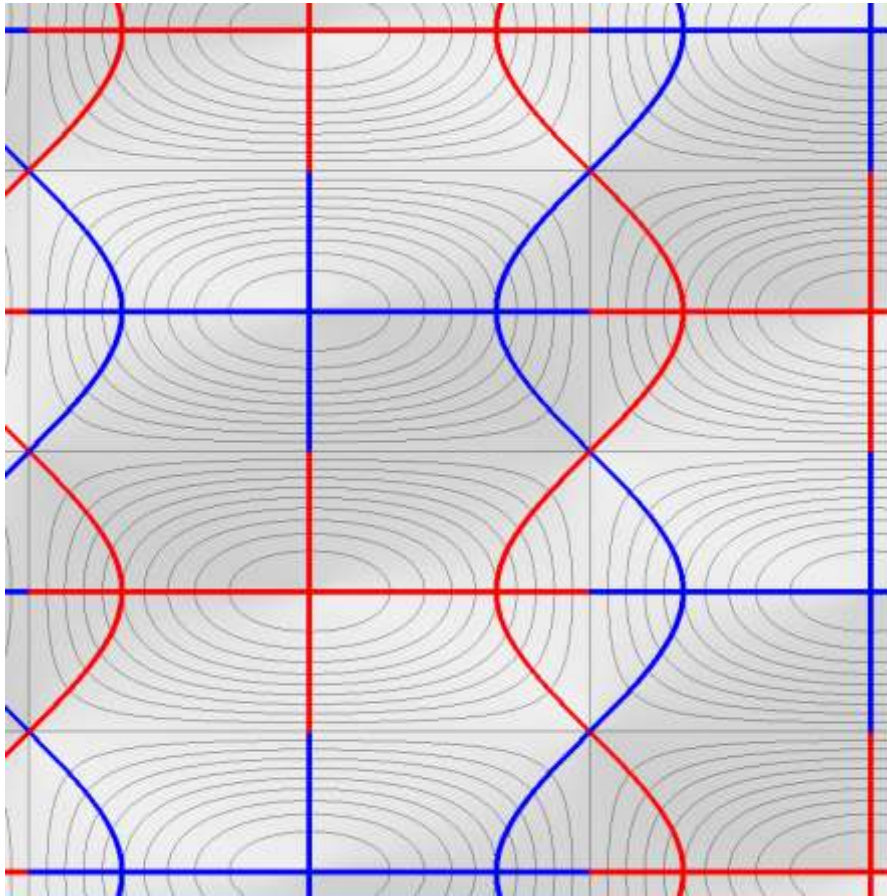
- **Again, valleys** are defined in complete analogy

Raw Features

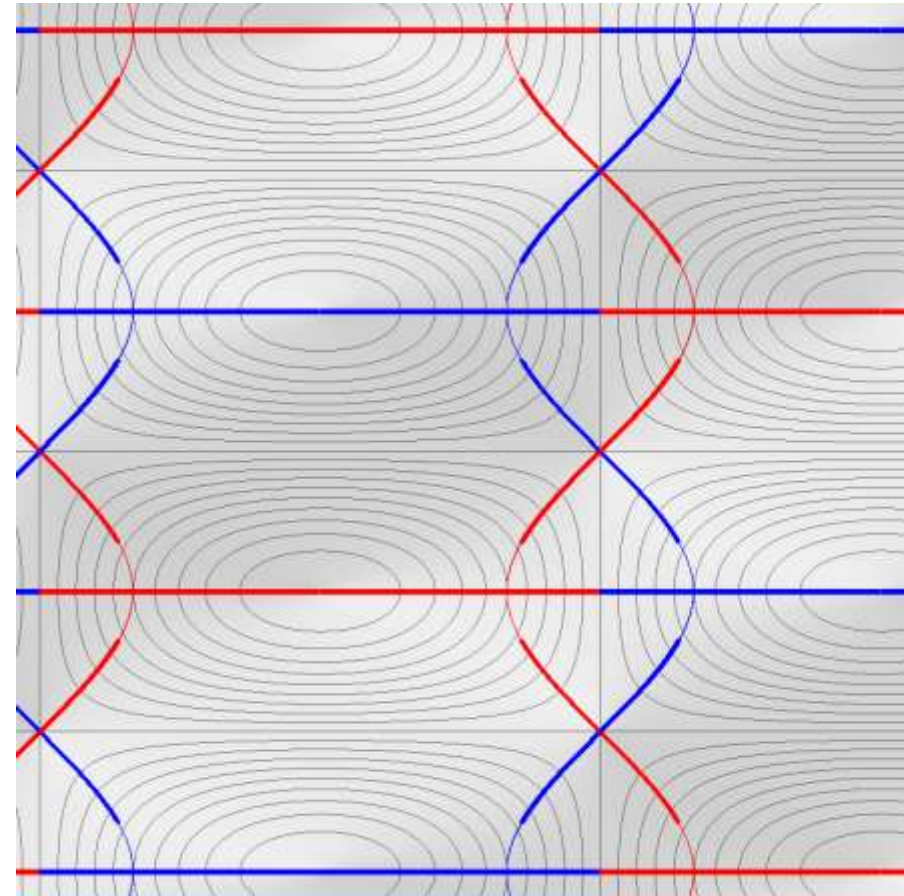
Definition used by [Damon 1999] and [Peikert/Sadlo 2008]:

- “Raw features” are points where $\mathbf{g} \cdot \mathbf{e}_i = 0$ for $(n-k)$ different Hessian eigenvectors $\mathbf{e}_i$
 - Necessary condition for height ridges and valleys
 - Checking the sufficient conditions (signs and order of eigenvalues) amounts to filtering the raw features
 - Can be used to define curves that connect ridges and valleys
 - Permits additional types of filters
 - max. angle between gradient and tangent of ridge line
 - minimum/maximum scalar value for ridges/valleys

Example: Filtering Crease Lines in 2D



Raw **ridges** and **valleys**



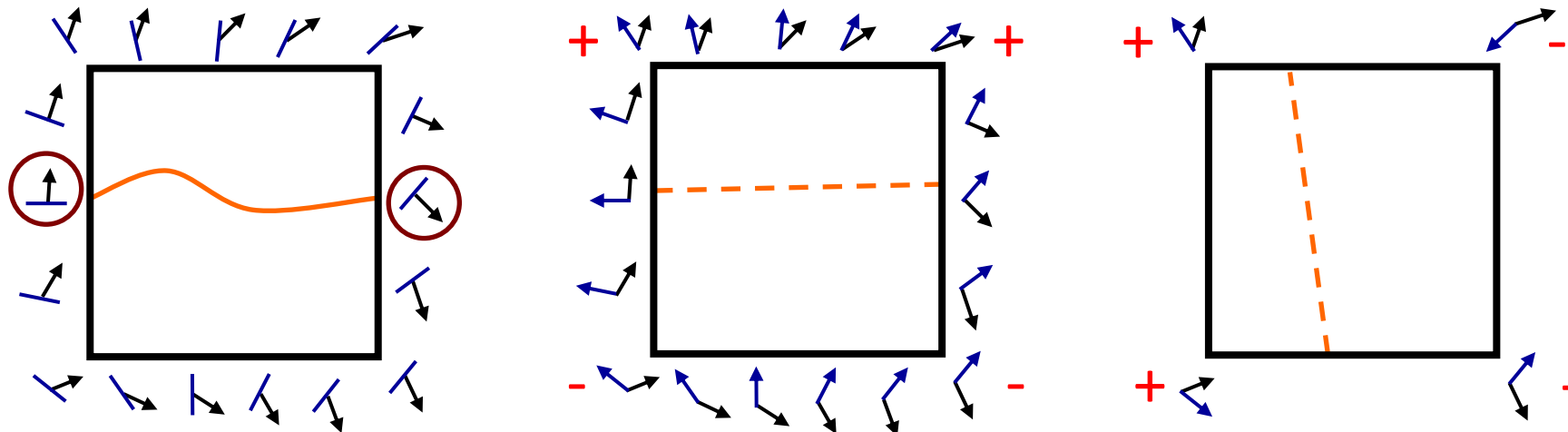
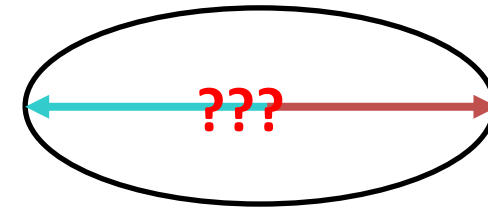
Filtered **ridges** and **valleys**:
Eigenvalues $\rightarrow$ non-conforming points removed
Angle to gradient $> 45^\circ \rightarrow$ thin lines

Raw Features Lines in 2D

- In 2D, **raw features lines** can be expressed as $\mathbf{g} \parallel \mathbf{H}\mathbf{g}$
 - Perpendicular to one eigenvector means parallel to the other
 - “Parallel Vector” operator from Chapter 13
 - [Peikert/Sadlo 2008]: In 2D, extract zero contour of $d = \det(\mathbf{g}|\mathbf{H}\mathbf{g})$
 - $\mathbf{g}|\mathbf{H}\mathbf{g}$ denotes a matrix with the first column $\mathbf{g}$ and the second column $\mathbf{H}\mathbf{g}$
 - **Idea:** should be able to use Marching Squares
 - Unfortunately, it’s not that easy. Before we find out why, let’s look at 3D...

Extracting Ridge Surfaces

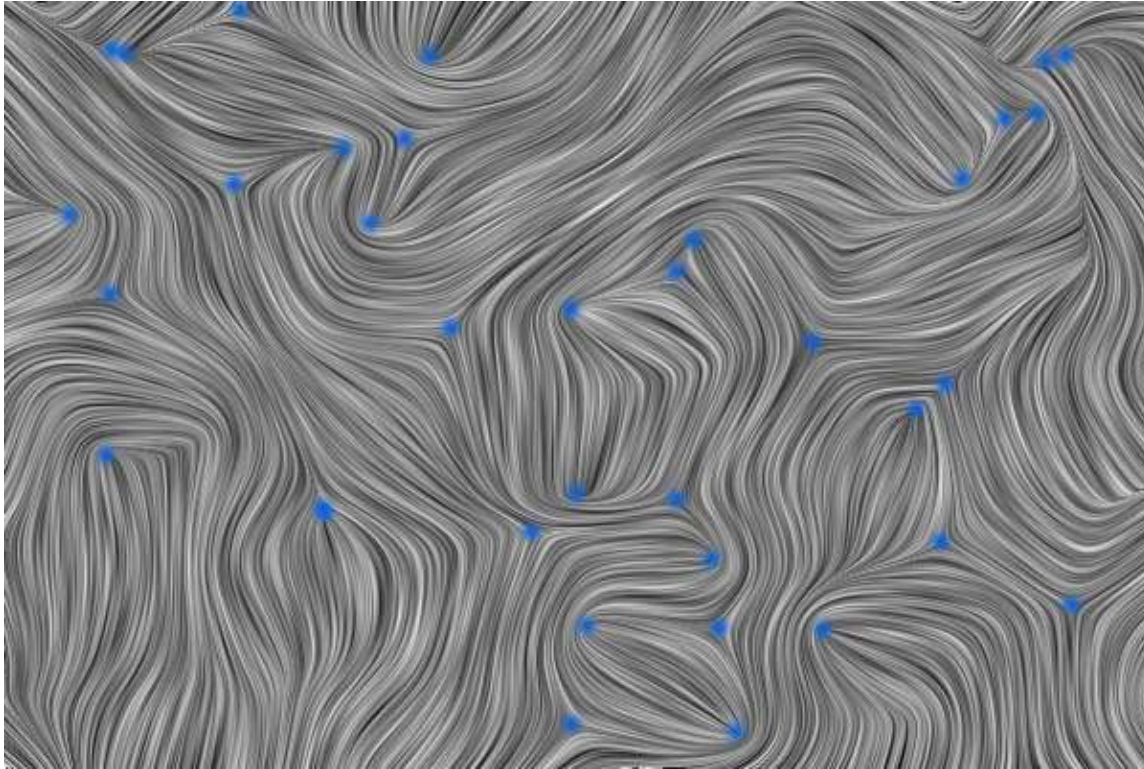
- Several works have used **Marching Cubes** to extract the zero-isocontour $\mathbf{g} \cdot \mathbf{e}_3 = 0$
- $\mathbf{e}_3$ is only defined up to its sign
 - We have to **orient** it
 - False orientation can introduce or cancel zero contours



Eigenvector Orientation: Approaches

- **Morse (1994): “Pairing of Eigenframes”**
 - Looks for the best overall eigenframe correspondence
- **Furst and Pizer (1996): “PCA-like”**
 - Averages outer products of eigenvectors at vertices
- **Kindlmann et al. (2006): “Evec tracking”**
 - Fine subsampling on the edge
 - Angular difference smaller than θ in each step

Aside: Tensor Field Topology



LIC rendering of
a major
eigenvector field

In tensor fields, **degenerate points** take over the role of critical points in vector field topology

- Places where tensor streamlines meet
- At least two eigenvalues are equal

Degenerate Points in 2D

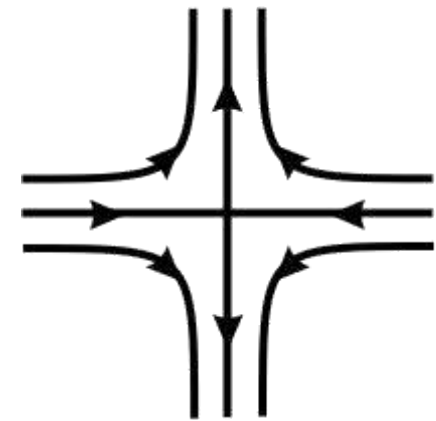
- Degenerate 2x2 symmetric matrices are a scalar multiple of the identity, $\mathbf{T}=\alpha\mathbf{I}$
- Two linear equations:
 - Zero off-diagonal: $T_{xy}=0$
 - Same value on diagonal: $T_{xx}-T_{yy}=0$
- In 2D tensor field: Leads to degenerate points

Poincaré Index of a Critical Point

Definition: Poincaré index of critical point

- Based on a closed curve around the critical point that is small enough not to include any other critical point
- Poincaré index is defined as the number of counterclockwise rotations of the vectors of $\mathbf{v}$ while traveling counterclockwise on the closed curve
 - in continuous vector fields,
index is a (possibly negative) integer

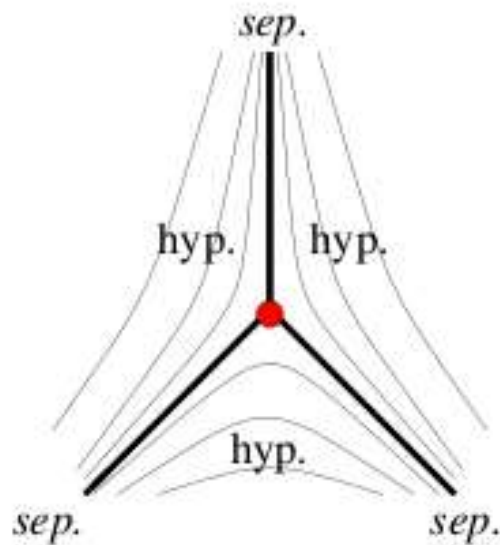
Example: $\text{index}(\text{saddle}) = -1$



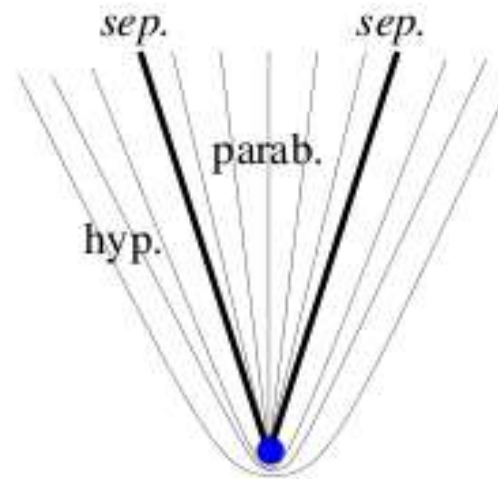
Saddle point
 $R_1 < 0, R_2 > 0$
 $I_1 = I_2 = 0$

Classifying Degenerate Points in 2D

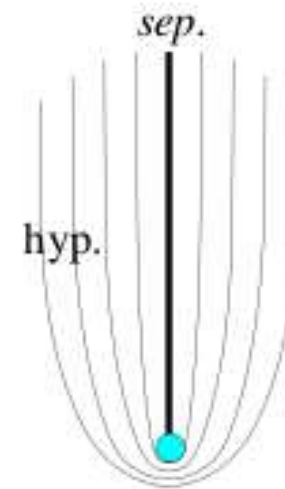
- Due to lack of orientation in eigenvectors, Poincaré index in eigenvector fields can be half-integer ($\pm 1/2$)
 - Leads to streamline behavior around degenerate points that would be impossible in vector fields:



TRISECTOR



WEDGE POINTS

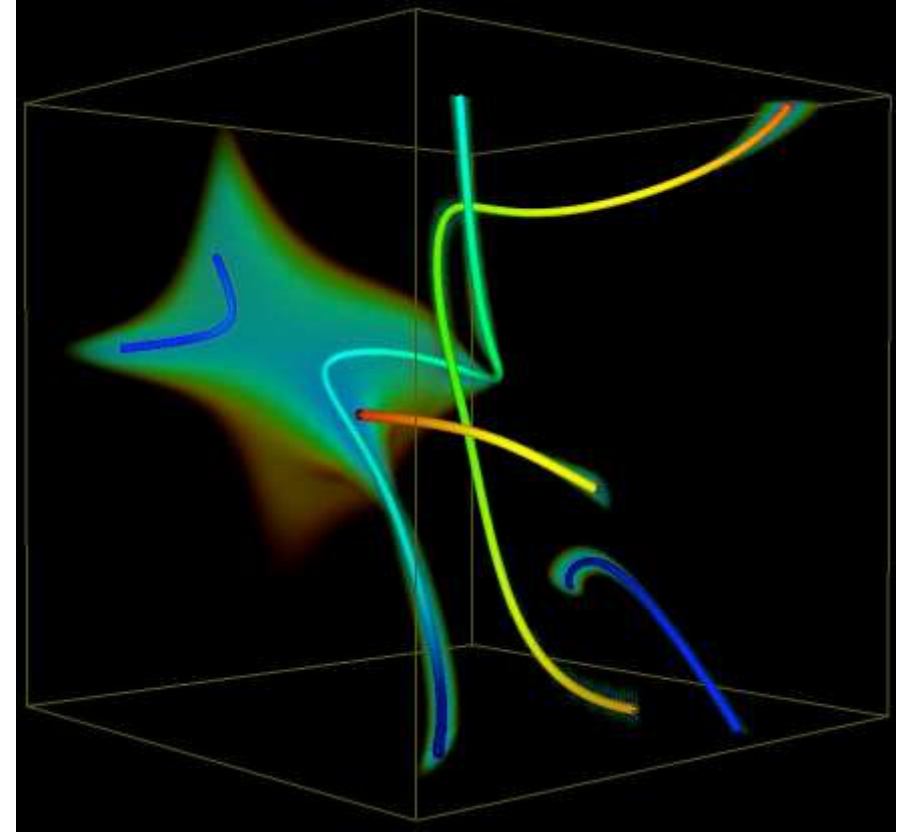
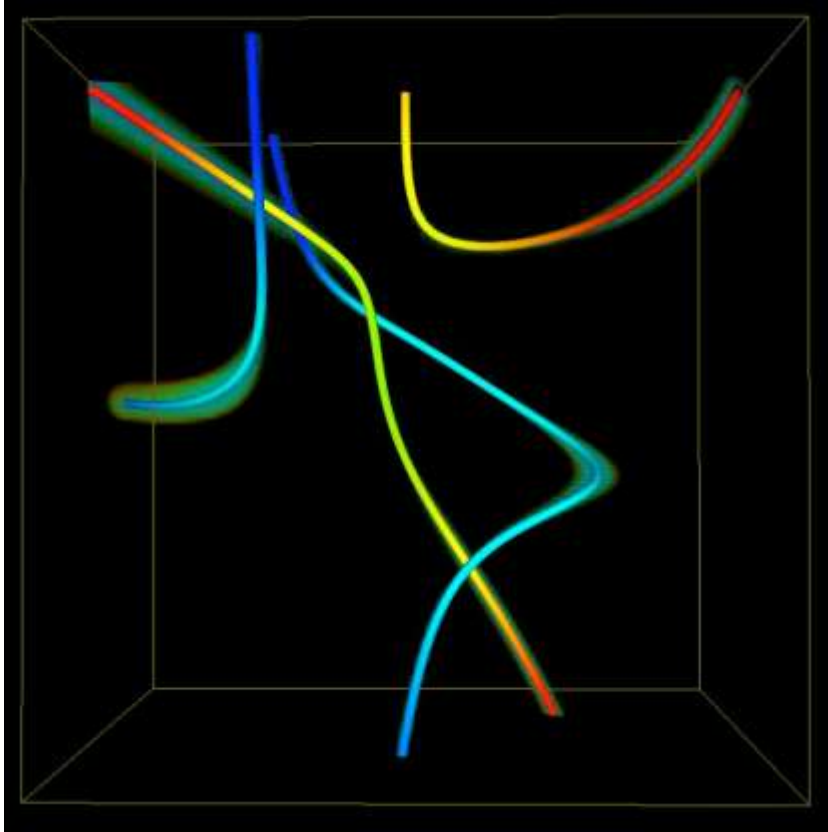


Degeneracies in 3D

- Types of degenerate 3x3 symmetric matrices:
 - Type P: Large pair of eigenvalues $\lambda_1=\lambda_2$
 - Type L: Small pair of eigenvalues $\lambda_2=\lambda_3$
 - Triple degeneracy: All eigenvalues $\lambda_1=\lambda_2=\lambda_3$
- In 3D tensor field: Leads to degenerate lines
 - Type P and Type L degeneracies form lines
 - *Intuition*: Transversal intersection of four-dimensional degenerate space and three-dimensional tensor field within six-dimensional space of all tensors
 - Triple degeneracies are structurally unstable
 - Vanish under arbitrarily small perturbations

Example: Degenerate Lines in 3D

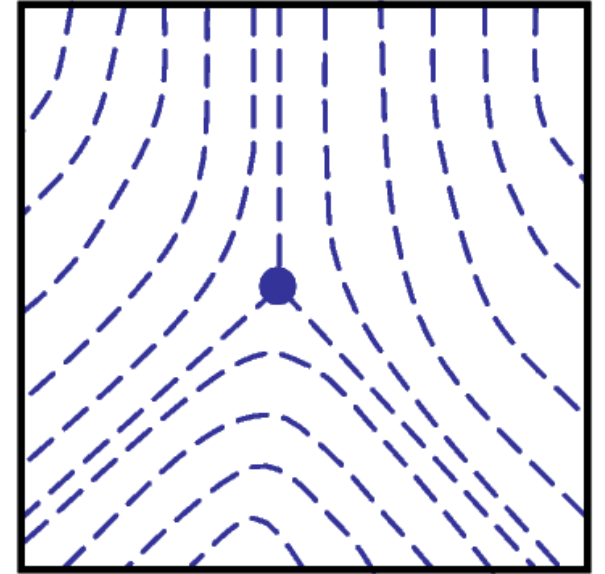
Degenerate Lines in Two Random Tensor Fields:



Warm colors: Type P
Cool colors: Type L

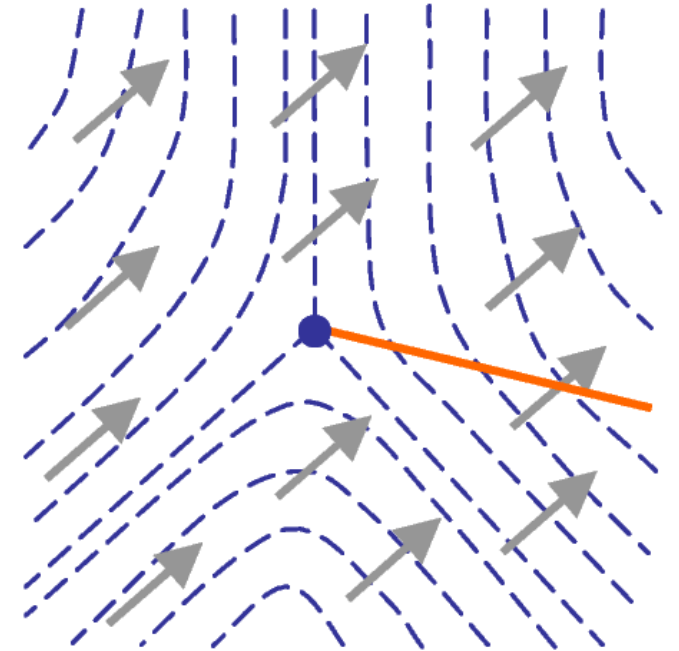
Non-Orientability of Eigenvectors

- **Fundamental problem:**
 - When the Hessian is degenerate ($\lambda_2 = \lambda_3$), trying to orient $\mathbf{e}_3$ leads to a **contradiction**
 - This happens along structurally stable lines in the volume
 - Expected under general conditions and stable under small perturbations



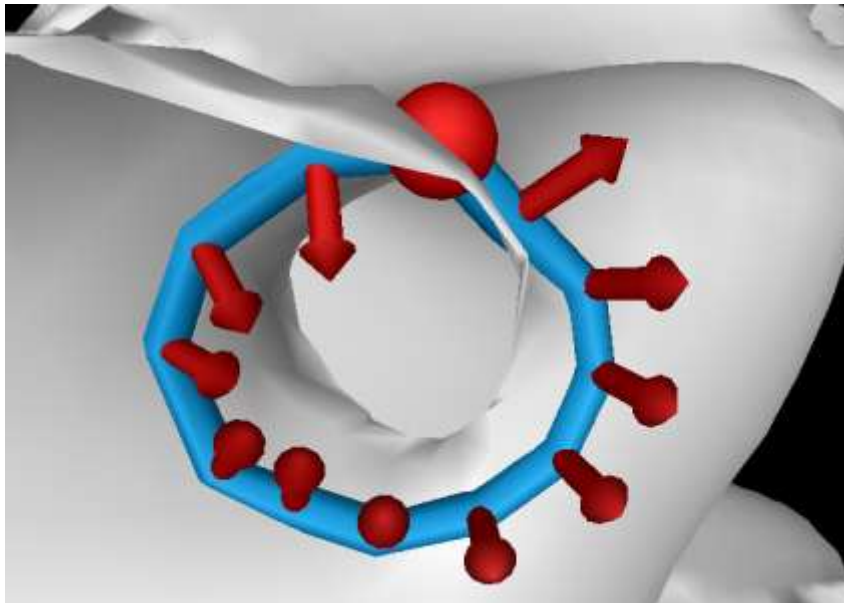
Consequences

- In 2D, crease lines **end in degenerate points**
- In 3D, crease surfaces **end in degenerate lines**
- Crease surfaces generally **do not branch**
 - In case of bifurcating structures, noise will cause the skeleton to break apart

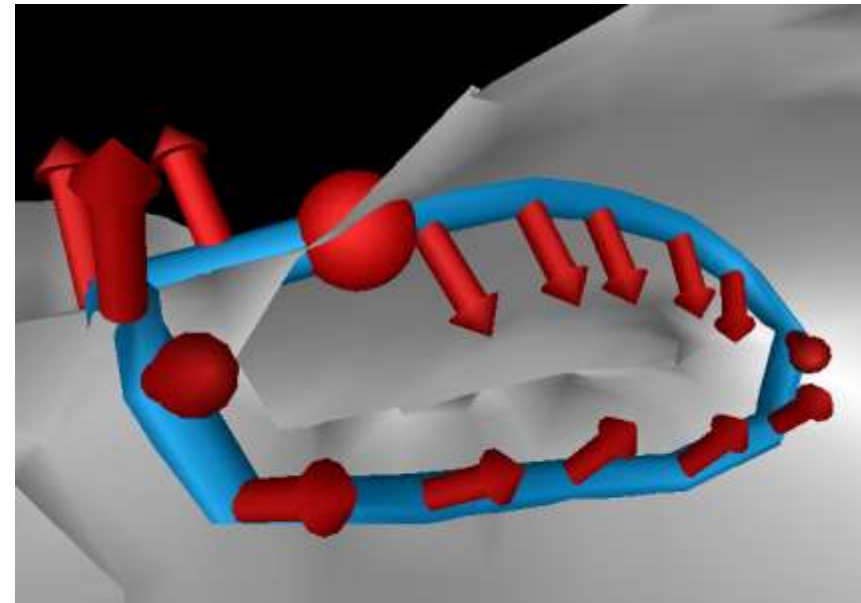


Creases are Non-Orientable

- $\mathbf{g} \cdot \mathbf{e}_3 = 0$ defines a **surface with boundary**
 - Surfaces with boundary need not be orientable
 - Previous authors report trouble when orienting their crease surfaces
 - We present the first examples:



Example 1



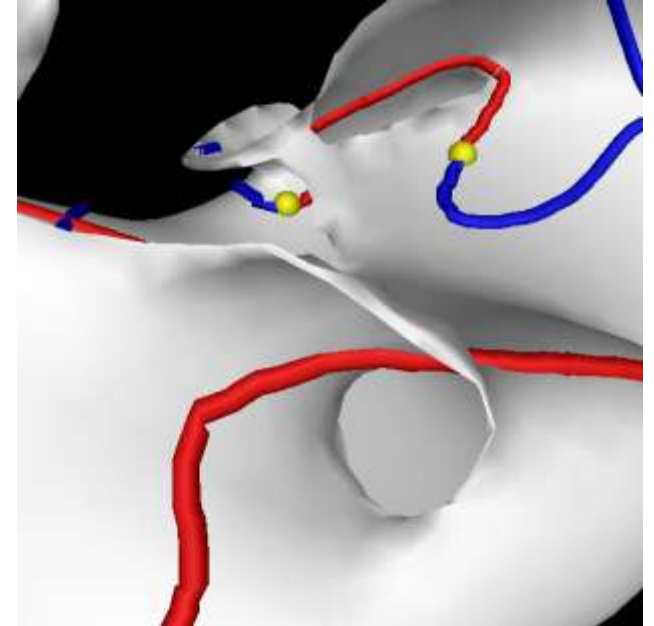
Example 2

But what about d ?

- $d = \det(\mathbf{g} \mid \mathbf{H}\mathbf{g} \mid \mathbf{H}\mathbf{H}\mathbf{g}) = 0$ when $\mathbf{g} \perp \mathbf{e}_3$
- **But:** $d=0$ also when $\mathbf{g} \perp \mathbf{e}_{1/2}$
 - When $\mathbf{g} \parallel \mathbf{e}_{2/1}$, both are true
 - The zero-isosurface of d self-intersects in the PV-lines $\mathbf{g} \parallel \mathbf{e}_i$

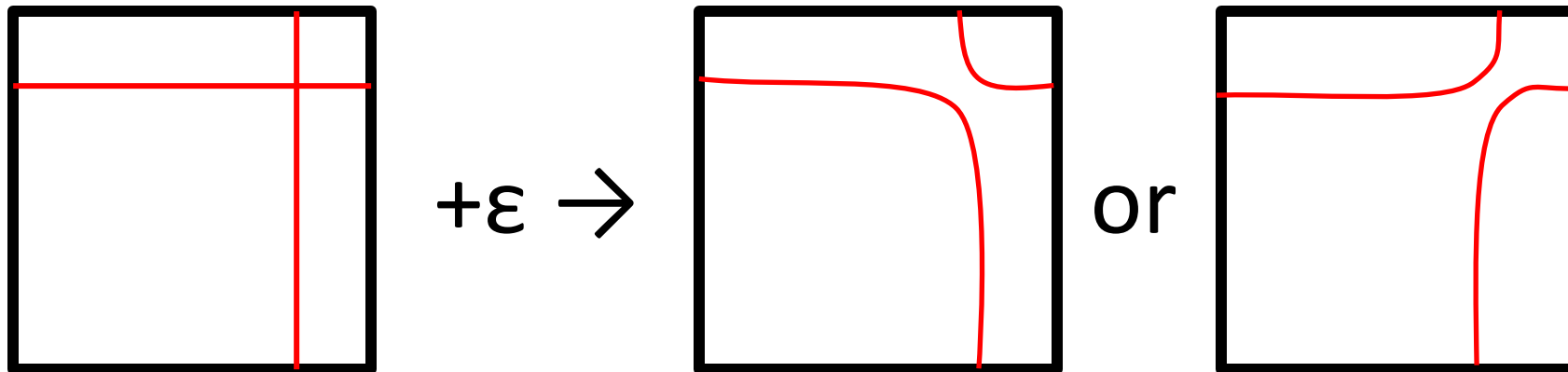
Consequence:

- Marching Cubes cannot reliably extract $d=0$, because it does not handle self-intersections



No Crossings in Marching Squares / Cubes

- Isocontours do not intersect “in general”:
 - If you assign random values to grid vertices, or extract a random isocurve, the chance of a self-intersection is 0.
 - At saddles, there is one pair of intersecting isocurves. However, the asymptotic decider breaks it apart using symbolic perturbation:

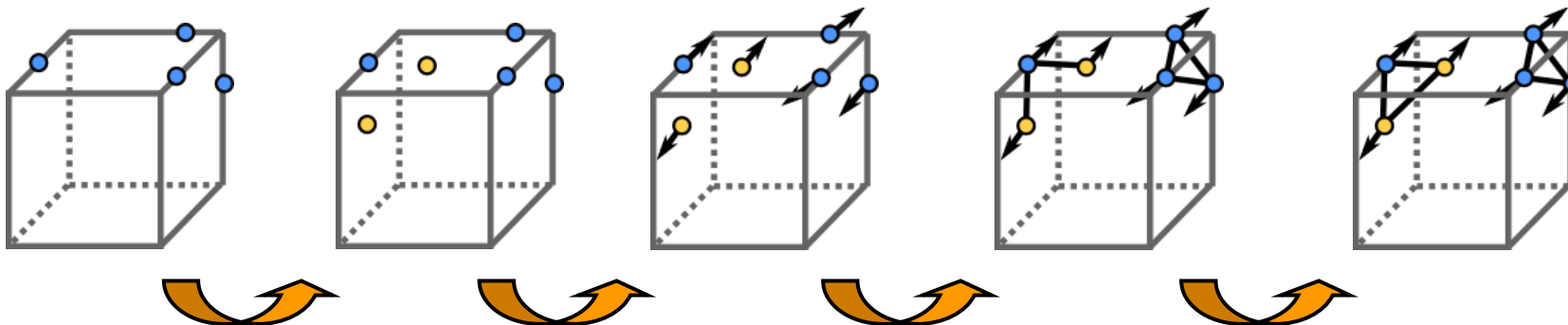


Summary: Theory of Height Ridge Extraction

- **Tensor topology** has consequences for the topology of crease surfaces
- Crease surfaces **end in degenerate lines**
 - Another type of boundary occurs when $\lambda_3/\lambda_1=0$
- Crease surfaces **are not orientable**
- **Marching Cubes case table is not suited for crease extraction**
 - Cannot deal with surfaces that end or self-intersect inside a voxel
 - Since crease surfaces are not orientable, we cannot close them and filter out the spurious parts later on
 - This would introduce self-intersections

Basic Idea Behind our Approach

- [Schultz et al. 2010]: Perform a cell marching, but avoid the (inappropriate) Marching Cubes case table
 - Find intersections between cell edges and surface
 - Find intersections between cell faces and boundary
 - Estimate surface normals
 - Based on them, connect the points on each cell face
 - Connect the boundary points for each cell



How we Solve the MC Problems

- We do not try to **orient eigenvectors**
- We allow for **boundaries within cells**
- **Surface orientability** is a global property
 - If we make per-cell decisions correctly, (non-) orientability of the final surface follows automatically
- **Rendering** of non-orientable surfaces:
 - Discard sign of normal in the lighting computation
 - Implemented with a shader program
- *Note:* For simplicity, we will explain ridge surface extraction. Valley surfaces can be extracted in analogy.

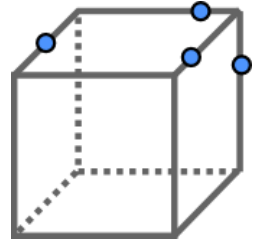
The Transformed Hessian Field $\mathbf{T}$

- From the Hessian $\mathbf{H}$, we define a differentiable symmetric tensor field $\mathbf{T}$ so that $\mathbf{T}\mathbf{g}$ projects the gradient $\mathbf{g}$ to the plane spanned by $\mathbf{e}_1$ and $\mathbf{e}_2$

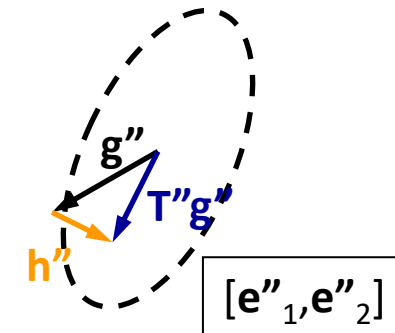
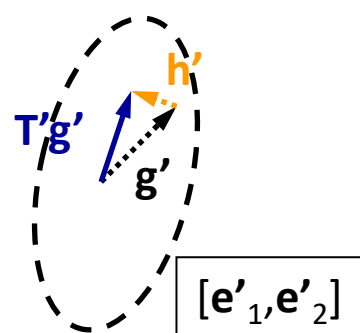
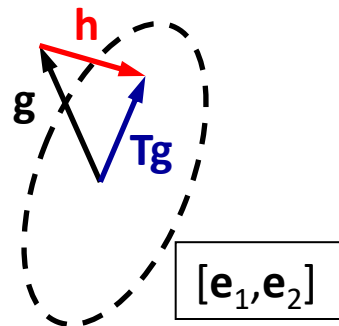
$$\mathbf{e}'_i := \mathbf{e}_i \quad \lambda'_1 := 1 \quad \lambda'_2 := 1 \quad \lambda'_3 := \begin{cases} 0 & \text{if } (\lambda_2 - \lambda_3) > \theta \\ \left(1 - \frac{\lambda_2 - \lambda_3}{\theta}\right)^2 & \text{else} \end{cases}$$

- Piecewise definition of λ'_3 takes care of the degenerate case $\lambda_2 = \lambda_3$ in a smooth manner

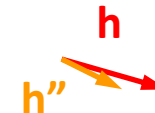
Finding Edge Intersections



- Consider $\mathbf{h} := \mathbf{Tg} - \mathbf{g}$
 - $|\mathbf{h}| = 0$ if and only if we are on the crease ($\mathbf{g} \perp \mathbf{e}_3$)
 - Otherwise, $\mathbf{h}$ specifies how $\mathbf{g}$ moves when being projected to the plane spanned by $\mathbf{e}_1/\mathbf{e}_2$
- Assume an edge intersection when $\mathbf{h} \cdot \mathbf{h}' < 0$
 - Does **not** orient $\mathbf{e}_3$, only uses orientation of $\mathbf{g}$
 - Estimate point of intersection based on relative magnitudes

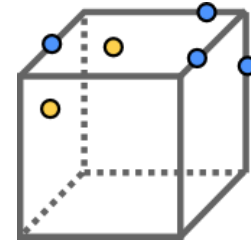


Intersection



No Intersection

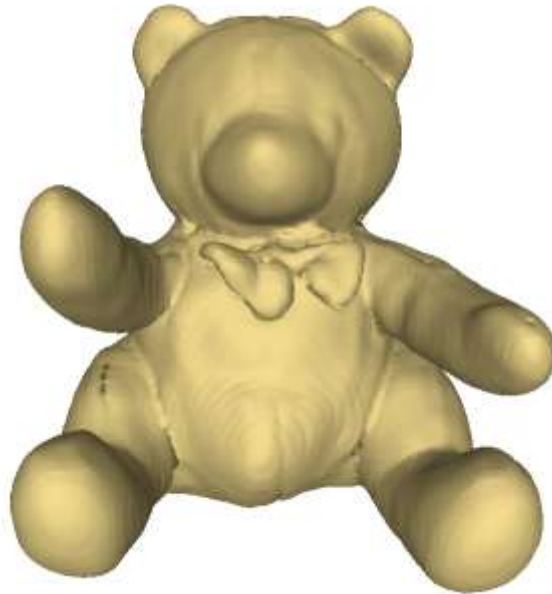
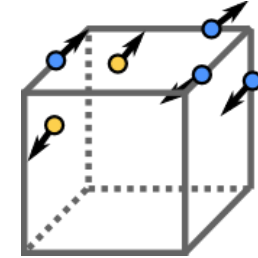
Finding Face Intersections



- Crease ends where $\lambda_2 = \lambda_3$ (**degeneracy**)
 - Existing algorithm by Zheng and Pang (2004) can be used to localize these points on cell faces
- Each face needs an **even number** of (edge+face) intersection points
 - Creases should either end in the cell or leave it
 - Only look for degeneracy on faces whose boundary is intersected an odd number of times
 - If degeneracy is not found, extract intersections on refined edges
 - *Note:* Unlike Marching Cubes for isosurfaces, we cannot guarantee finding all intersections; this remains an open research question

Estimating Normals

- Recall that $\mathbf{h} := \mathbf{T}\mathbf{g} - \mathbf{g} \equiv \mathbf{0}$ on the crease
 - Tangential to the crease, $\partial \mathbf{h}$ vanishes
 - Jacobian $\nabla \mathbf{h}$ has rank one
 - Eigenvector of $\nabla \mathbf{h}$ for only non-zero eigenvalue in normal direction



Normals from mesh

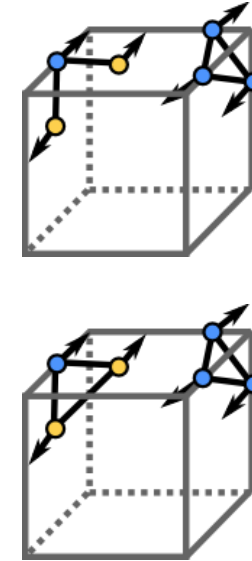


Normals from $\nabla \mathbf{h}$

Selecting Topology

Topology selected after enumerating all options:

1. Connect points pairwise per-face
 - Avoid self-intersections
 - Try to agree with extracted normal
2. Connect degenerate points per-cell
 - Similar conditions
 - Always an even number per cell



- Sharing faces between adjacent cells ensures that we get a consistent final mesh

Example at a Degenerate Line

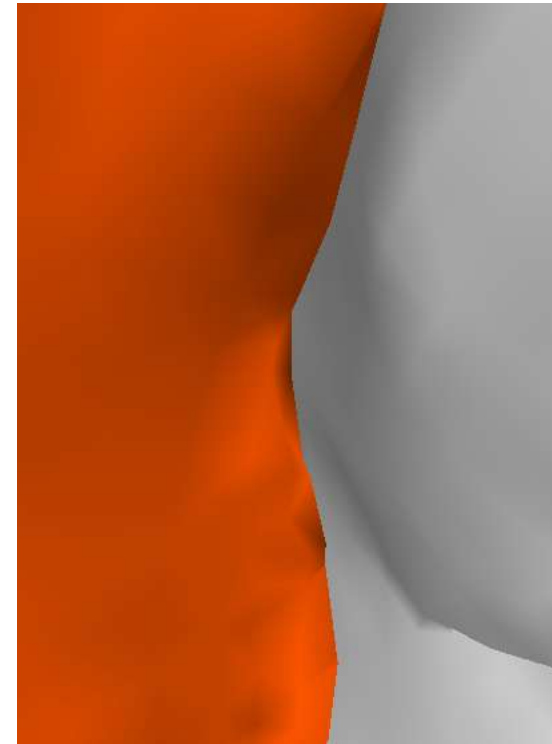
- Orange surface terminates just before it would meet the grey surface in a T-junction



MC with PCA



MC with PCA
at double resolution



Proposed Method

Experimental Setup

- How important is it **in practice** to get this right?
- *Example*: CT scan of a teddy bear
 - Due to different materials, the bear is difficult to extract as an isosurface
 - Therefore: Attempt to extract it as a ridge surface
- Creases always require **post-filtering**
 - Require that $|\mathbf{g}|/|\lambda_3| < \Theta$
 - *Intuition*: Relative to the slope of the ridge, the curvature perpendicular to it should be sufficiently high
 - Used connected component analysis to remove an unwanted background object

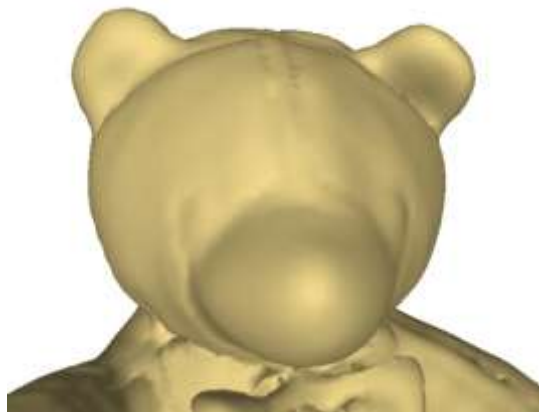
Qualitative Results



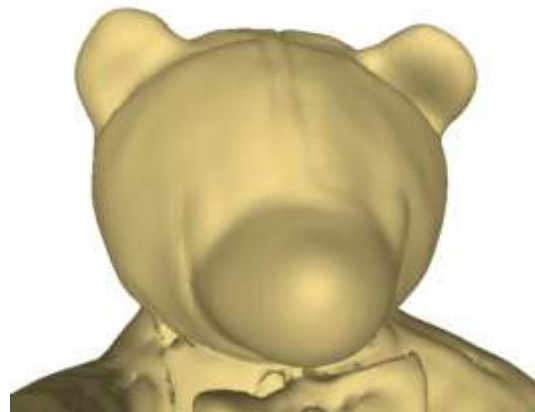
Eigenvector
Tracking



Eigenvector PCA



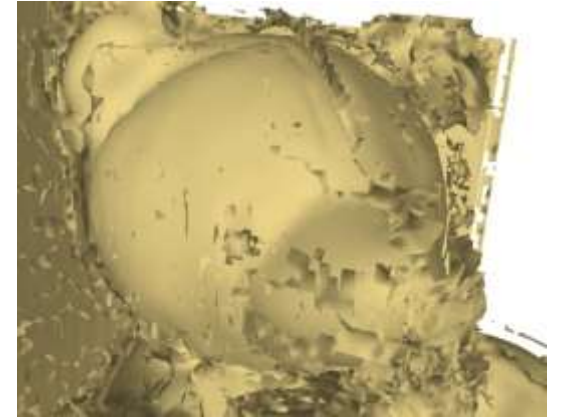
Proposed Method



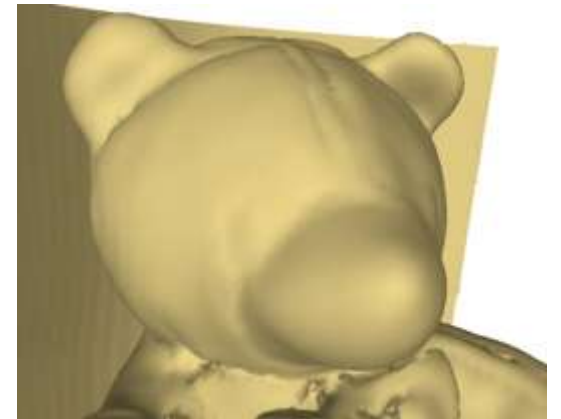
Eigenvector tracking
at double resolution

Impact of Adaptive MC Surprisingly Large?

- Why are MC results so bad even in places where the ridge *is not supposed to end*?
- Why does refining the grid help, if MC does not respect Hessian degeneracies *in principle*?
- Answer: **Noise**
 - Spurious ridges run through the same cell as the desired surface
 - In a refined grid, this becomes less likely



Before Post-Filtering



After Post-Filtering

Why not simply use Adaptive MC?

- Adaptive grid refinement around the surface proposed by Sadlo and Peikert (2007)
 - Clear improvement over non-adaptive MC, but:
 - Overtesselation and big computational effort
 - Does not solve the problem at real boundaries

Method	Time (s)	# triangles
even tracking	24+1	66,662
even tracking (2× res.)	180+4	578,489
PCA	18+2	201,068
proposed method	38+3	246,040

Quantitative Results

- Estimate accuracy of the extracted surface
 - Sample a large number of points from the surface
 - Perform a gradient descent to the crease ($|\mathbf{h}|^2 \rightarrow \min$)

Method	area	error (mm)
evec tracking	0.21 m ²	0.02 (0.14)
PCA	0.49 m ²	0.07 (0.31)
proposed method	0.47 m ²	0.02 (0.13)

- Quantify small holes in the resulting mesh

Method	Bdy components of length				
	3	4	5	6	> 6
evec tracking	2158	59	408	69	353
evec tracking (2× res.)	2277	97	469	80	533
PCA	44	31	20	67	923
proposed method	0	10	0	0	136

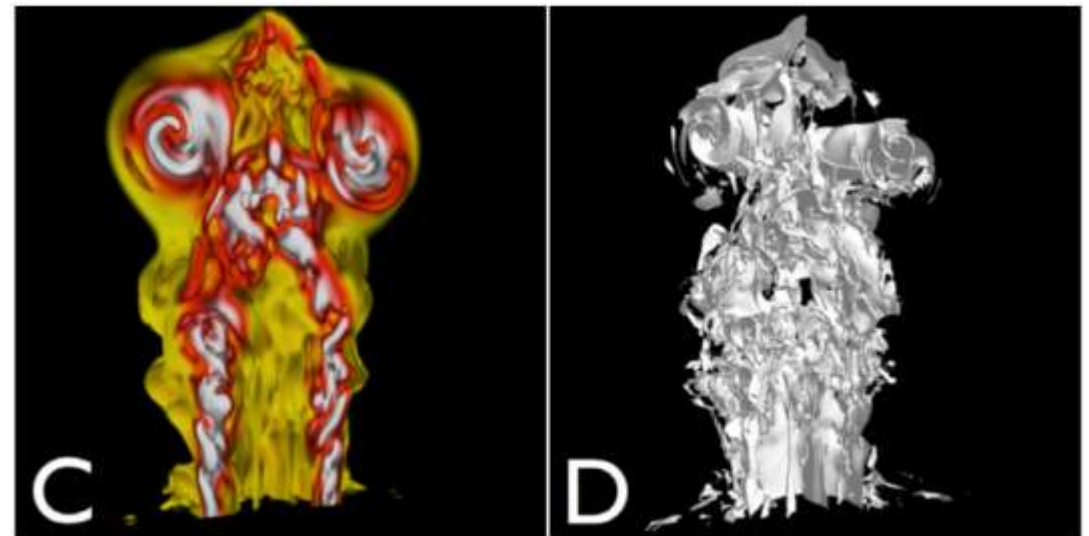
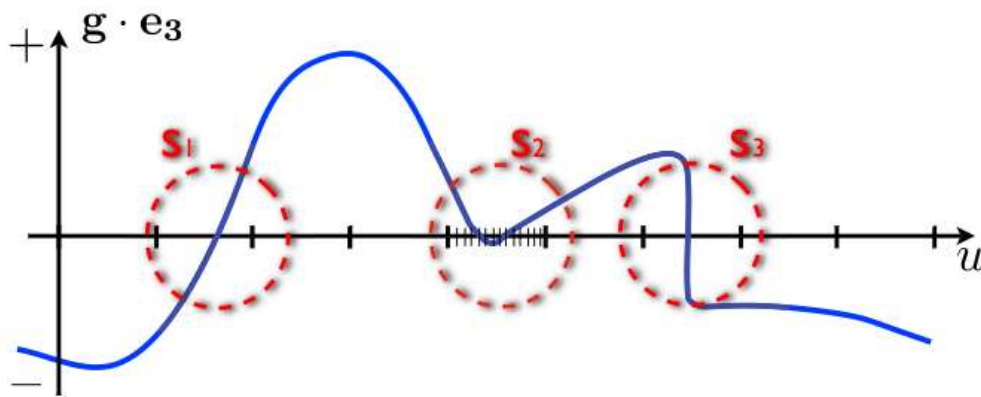
Summary: Method for Height Ridge Extraction

- Transformed Hessian $\mathbf{T}$ provides a unified framework for:
 - Finding edge intersections without having to orient $\mathbf{e}_3$
 - Estimating surface normals
- Efficient and Reliable Method for Crease Surface Extraction
 - Addresses problems raised by theoretical analysis
 - Validated qualitatively and quantitatively

Ray Tracing Ridge Surfaces

Image-based approach by [Barakat/Tricoche 2010]:

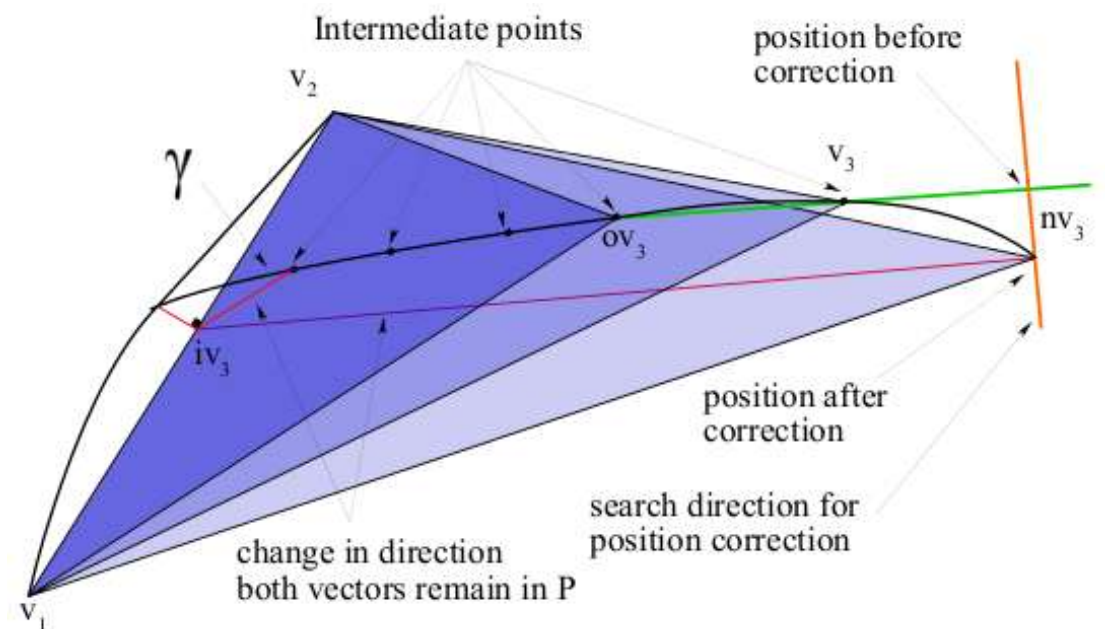
- Localize zero crossings of $\mathbf{g} \cdot \mathbf{e}_3$ along a viewing ray
- Use adaptive stepsize to consistently orient $\mathbf{e}_3$
- Use space leaping to avoid computation in regions where eigenvalue constraints or other filters (scalar value etc.) are violated



Alternating Sampling and Meshing

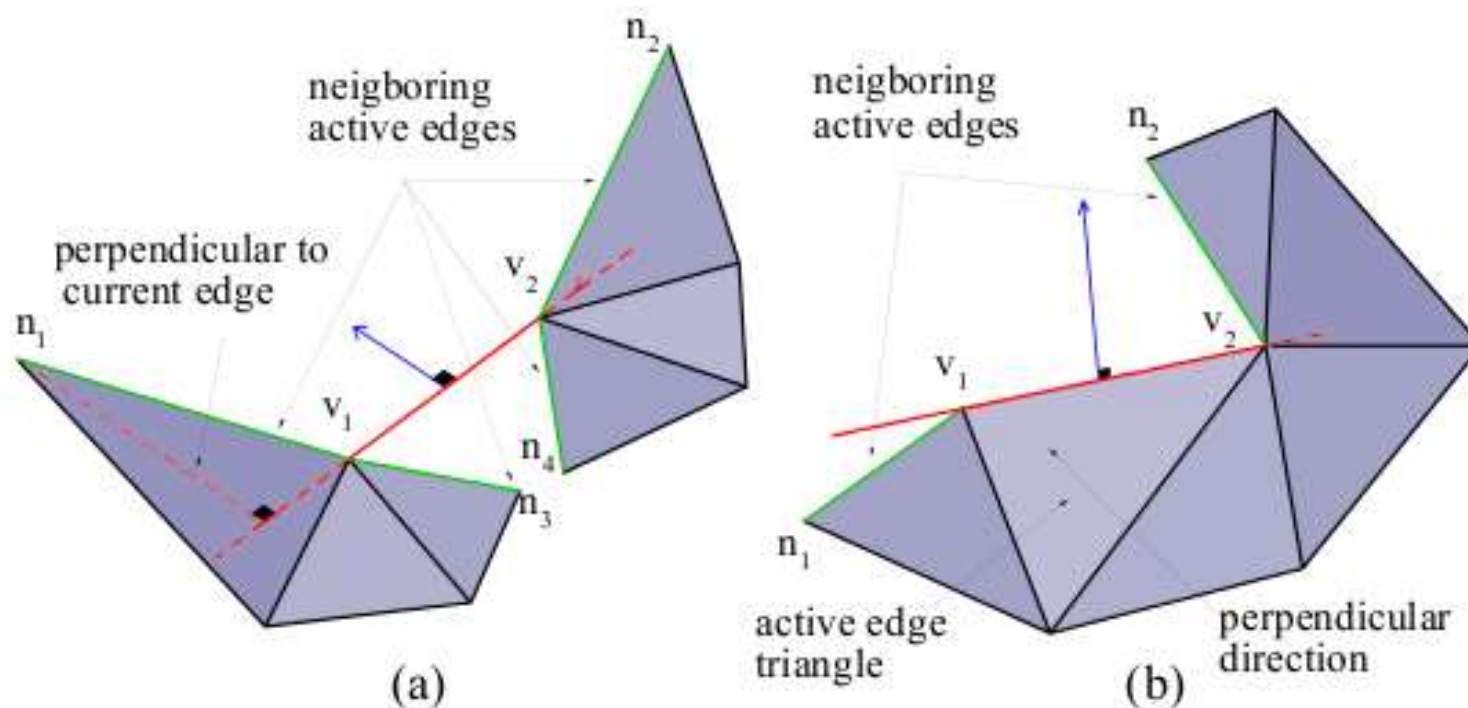
Alternative approach by [Barakat et al. 2011]:

- Instead of processing cell by cell, grow the surface from a seed triangle
- Grow new triangles in very small steps to stay on the same ridge and avoid meshing errors
 - GPU-based sampling to compensate effort
 - Still no formal guarantees



Challenges in Front Propagation

Compared to cube marching, front propagation produces higher-quality triangles, but requires dealing with some non-trivial situations, such as merging fronts:



Outlook: Ridges in Scale Space

Example results that show the impact of scale (width of ridges) in lung CT and brain MRI data



Summary: Height Ridges and Valleys

- **Height ridges and valleys** provide a local feature definition of crease lines and surfaces
 - Based on orthogonality between gradient and specific Hessian eigenvectors
- **Non-orientability** of eigenvectors and of crease surfaces poses a challenge for crease extraction algorithms
- We introduced a state-of-the-art **cell marching algorithm** for crease surface extraction and briefly discussed alternative approaches

References

- David Eberly: Ridges in Image and Data Analysis. Kluwer Academic Publishers, 1996
- Ronald Peikert, Martin Roth: The “Parallel Vectors” Operator – A Vector Field Visualization Primitive. Proc. IEEE Vis 1999, pp. 263-270
- Ronald Peikert, Filip Sadlo: Height Ridge Computation and Filtering for Visualization. Proc. PacificVis 2008, pp. 119-126
- Thomas Schultz, Holger Theisel, Hans-Peter Seidel: Crease Surfaces: From Theory to Extraction and Application in Diffusion Tensor MRI. IEEE TVCG 16(1):109-119, 2010
- Samer Barakat, N. Andryscio, Xavier Tricoche: Fast Extraction of High-Quality Crease Surfaces for Visual Analysis. Computer Graphics Forum 30(3):961-970, 2011

Announcements

- Discussion of trial exam on July 15 @ 14 c.t.
 - Several students still need to present to be admitted to the exam.
 - Please contact me TODAY if this affects you!
- Q&A session during lecture slot on Wed, July 22 @ 10 c.t.
- **Final exam on Monday, August 3 @ 12:00 sharp in Meckenheimer Allee 176, HS IV**
 - An A4 cheat sheet (both sides, in your own handwriting) is permitted
- Repeat exam on Mon, September 21 @ 11:00 in Meckenheimer Allee 176, HS II

Exam-like Questions

1. State an advantage of the height ridge definition compared to the watershed based definition of ridges.
2. What are two important differences between isosurfaces and ridge surfaces in a scalar volume?
3. One strategy for ridge extraction is to first compute raw features, then filter them to only retain relevant ridges. Describe one filtering criterion that can be used for this.